Design Patterns Course

Learn, Build, Adapt

Session #7

2024

Irina Kamalova

VP Lead Software Engineer @ JPMC
ex-Revolut & ex-Yandex
Built systems to support 45M+ clients

Refreshing

- What **Iterator Pattern** is useful for?
- What **Composite Pattern** is useful for?

07.1

State

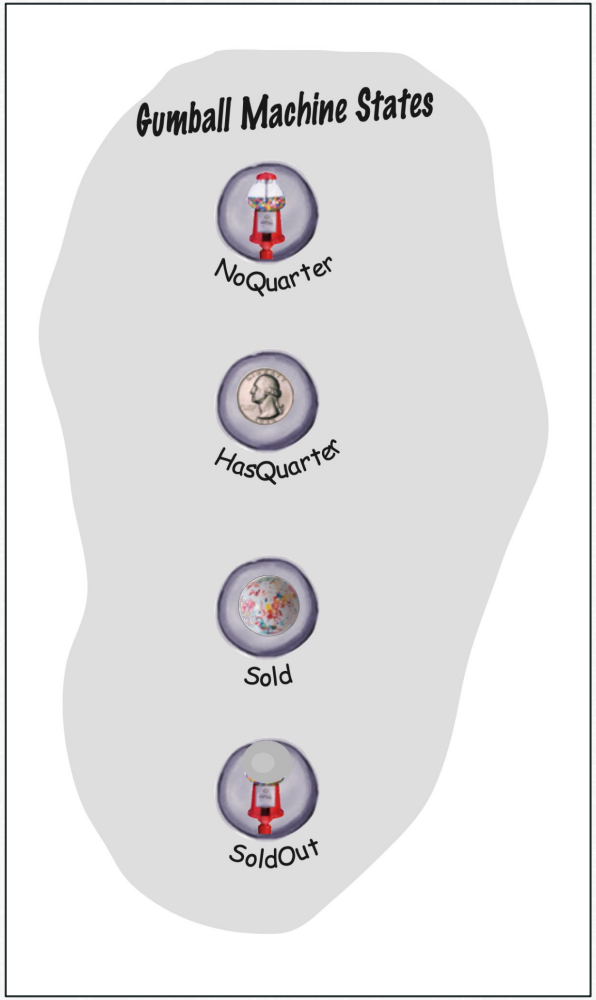
Behavioral

Behavioral Patterns

- Chain of Responsibility
- Command
- Iterator
- Mediator
- Memento
- Observer
- → State
- Strategy
- Template Method
- Visitor

We **create** a **pattern** for some **behavior** from scratch.

Example


Gumball Machine States
NoQuarter
HasQuarter
Sold
SoldOut

State

The State Pattern allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

Let's consult with Raccoon

https://refactoring.guru/design-patterns/state

Java

```
enum State {  3 usages
    LOCKED,  no usages
    PLAY,  no usages
    READY  no usages
}
```

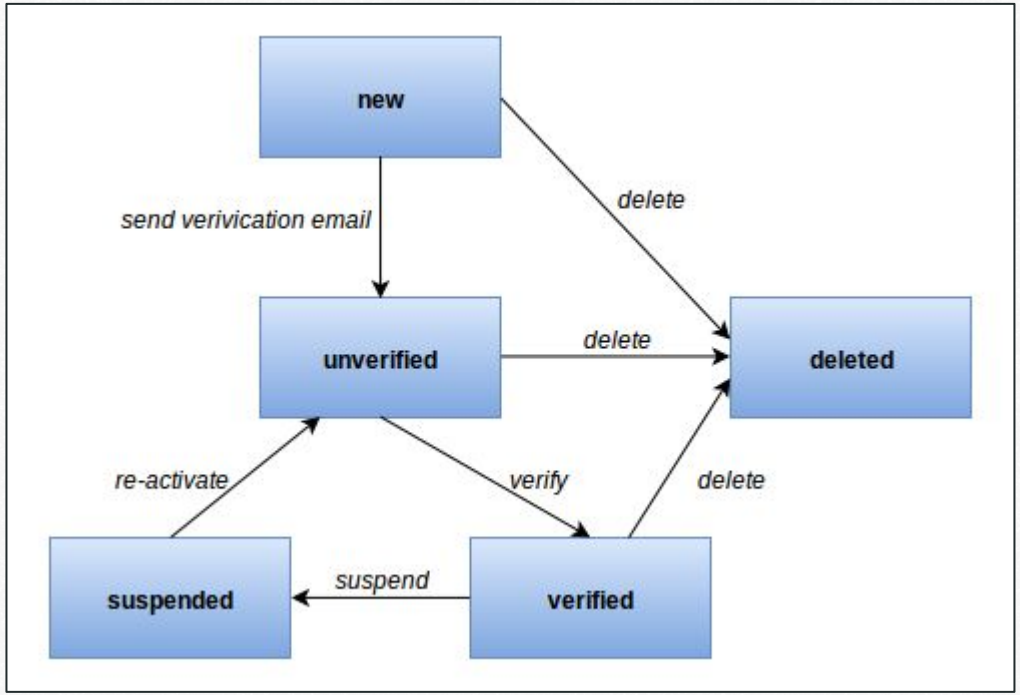
Java

```java
static class Player {  2 usages
    private State state;  12 usages
    enum State {  12 usages
        LOCKED,  2 usages
        PLAY,  4 usages
        READY  3 usages
    }

    public String onLock() {  no usages
        if (state == State.PLAY) {
            this.setCurrentTrackAfterStop();
        }
        state = State.LOCKED;
        return "Locked";
    }

    public String onPlay() {  no usages
        if (state == State.LOCKED) {
            state = State.READY;
            return "Ready";
        } else if (state == State.READY) {
            state = State.PLAY;
            return this.startPlayback();
        } else { // (state == State.PLAY)
            state = State.READY;
            return "Paused...";
        }
    }

    public String onNext() {  no usages
        if (state == State.PLAY) {
            return this.nextTrack();
        } else return "Locked";
    }
    public String onPrevious() {  no usages
        if (state == State.PLAY) {
            return this.previousTrack();
        } else return "Locked";
    }
```

Java

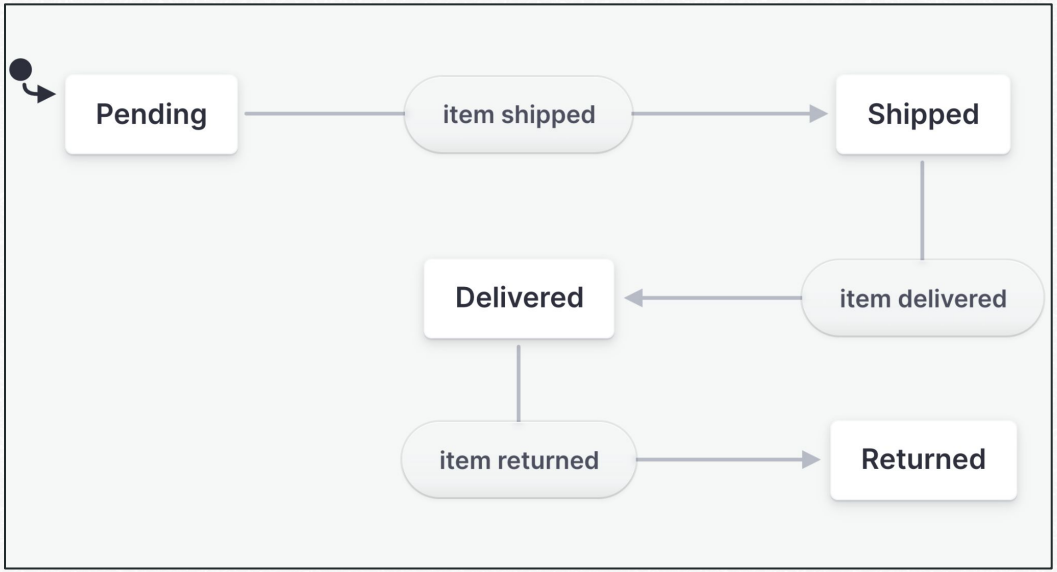
```java
static class Player {  6 usages
    private State state;  10 usages
    enum State {  11 usages
        LOCKED,  1 usage
        PLAY,  5 usages
        READY  2 usages
    }

    public String onPlay() {  1 usage
        if (state == State.PLAY) {
            state = State.READY;
            return "Paused...";
        } else {
            state = State.PLAY;
            return this.startPlayback();
        }
    }

    public String onLock() {  1 usage
        if (state == State.PLAY) {
            this.setCurrentTrackAfterStop();
        }
        state = State.LOCKED;
        return "Locked";
    }

    public String onNext() {  1 usage
        if (state == State.PLAY) {
            return this.nextTrack();
        } else return "Locked";
    }
    public String onPrevious() {  1 usage
        if (state == State.PLAY) {
            return this.previousTrack();
        } else return "Locked";
    }
```

The Finite State Machine



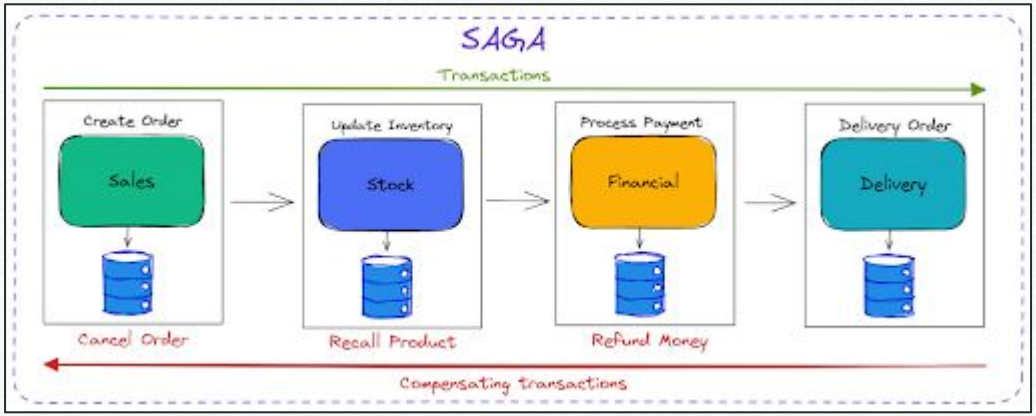
Source

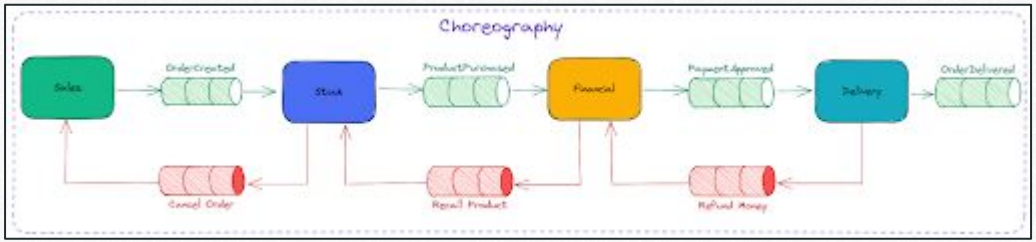
The Finite State Machine



Source

System Design World





In your experience…

```
enum State {  3 usages
    LOCKED,  no usages
    PLAY,  no usages
    READY  no usages
}
```

…have you seen it?

Isn't it CREATIONAL pattern/concept?

```
enum State {  3 usages
    LOCKED,  no usages
    PLAY,  no usages
    READY  no usages
}
```

The Finite State Machine

…Don't you think **you meet** this concept **everyday**?

07.2

Proxy

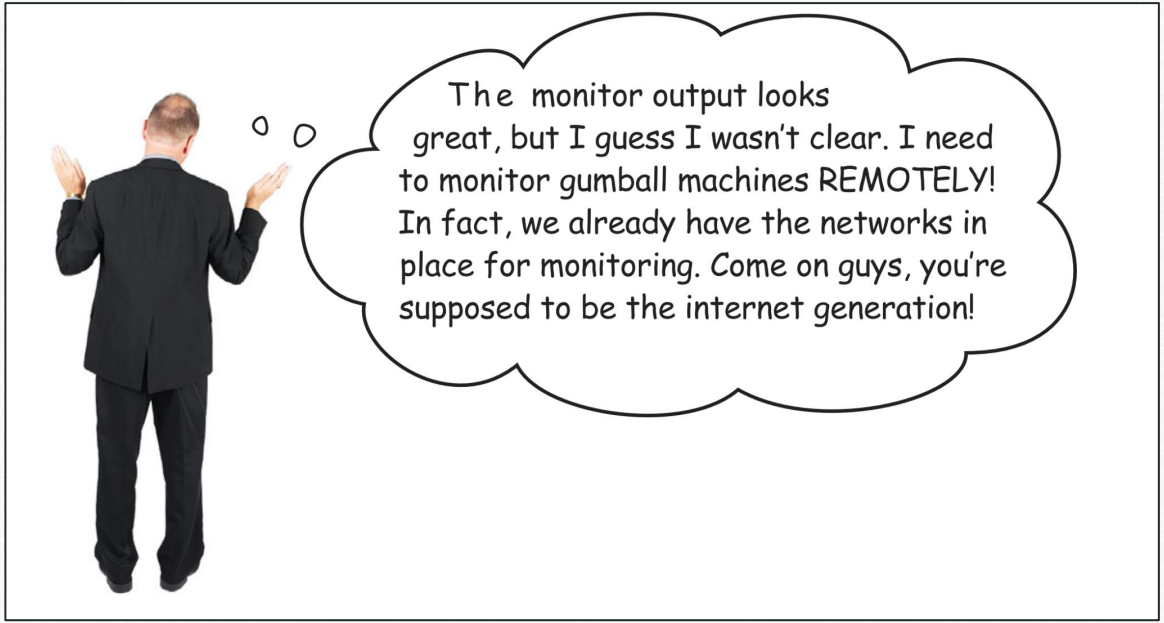
Structural

Structural Patterns

- Adapter
- Bridge
- Composite
- Decorator
- Facade
- Flyweight
- → Proxy

We don’t change the behavior of original action/actor/entity, but we **create** a **pattern** for the **structure** to add new behaviour or change the behavior by adding a new layer.

Example


The monitor output looks great, but I guess I wasn't clear. I need to monitor gumball machines REMOTELY! In fact, we already have the networks in place for monitoring. Come on guys, you're supposed to be the internet generation!

The Proxy Pattern

The Proxy Pattern provides a surrogate or placeholder for another object to control access to it.

Let's consult with Raccoon

https://refactoring.guru/design-patterns/proxy

More examples

Firewall Proxy controls access to a set of network resources, protecting the subject from "bad" clients.

Habitat: often seen in the location of corporate firewall systems.

Help find a habitat

Smart Reference Proxy provides additional actions whenever a subject is referenced, such as counting the number of references to an object.

Caching Proxy provides temporary storage for results of operations that are expensive. It can also allow multiple clients to share the results to reduce computation or network latency.

Habitat: often seen in web server proxies as well as content management and publishing systems.

More examples: Cache

Cache Loader

Let's build cache using CacheLoader. If we retrieve an element from cache and if the element is not available, CacheLoader will automatically get it for you from the source.

```
LoadingCache<String, User> userLoadingCache;

userLoadingCache = Caffeine.newBuilder()
                        .expireAfterWrite(1, TimeUnit.HOURS)
                        .maximumSize(100)
                        .build(key -> UserRepo.getUser(key));
```

UserRepo is a user repository class, which is responsible to interact with DB. Below class is just a template.

```
public class UserRepo {
    public static User getUser(String userId) {
        return new User(userId, "UserFromDB", "USER");
    }
}
```

Try to access the user object, before adding it.

```
//Get user from cache. If not available it will be fetched from DB.

User user = userLoadingCache.get("user_1");
if (user != null) {
   System.out.println(user.getName());
}
```

Source

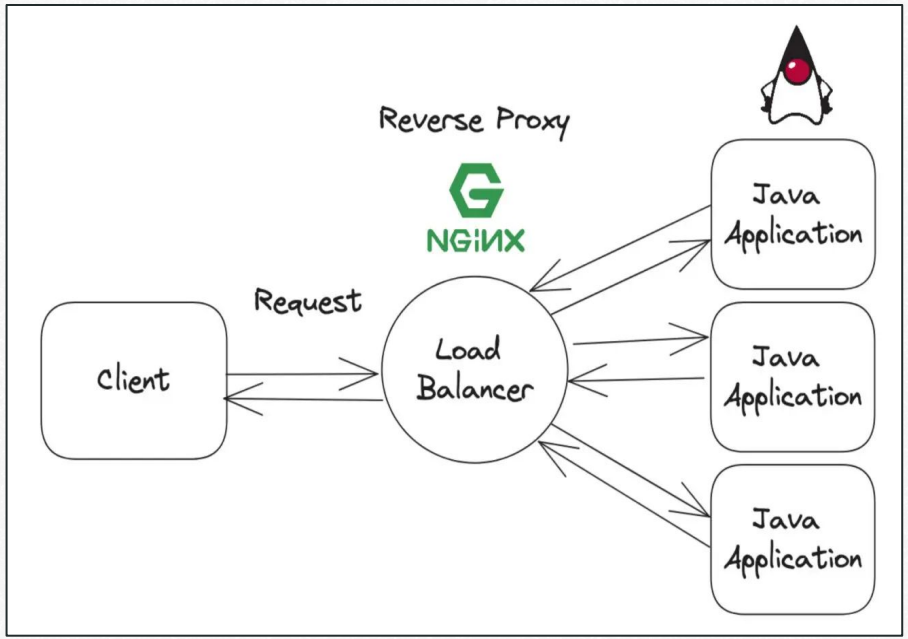
Python caching example

```python
# Function to get the HTML Content
def get_html_data(url):
    response = requests.get(url)
    return response.text


# Memoize function to cache the data
def memoize(func):
    cache = {}

    # Inner wrapper function to store the data in the cache
    def wrapper(*args):
        if args in cache:
            return cache[args]
        else:
            result = func(*args)
            cache[args] = result
            return result

    return wrapper


# Memoized function to get the HTML Content
@memoize
def get_html_data_cached(url):
    response = requests.get(url)
    return response.text


# Get the time it took for a normal function
start_time = time.time()
get_html_data('https://books.toscrape.com/')
print('Time taken (normal function):', time.time() - start_time)

# Get the time it took for a memoized function (manual decorator)
start_time = time.time()
get_html_data_cached('https://books.toscrape.com/')
print('Time taken (memoized function using manual decorator):', time.time()
```

```python
# Import the required modules
from functools import lru_cache
import time
import requests


# Function to get the HTML Content
def get_html_data(url):
    response = requests.get(url)
    return response.text


# Memoized using LRU Cache
@lru_cache(maxsize=None)
def get_html_data_lru(url):
    response = requests.get(url)
    return response.text


# Get the time it took for a normal function
start_time = time.time()
get_html_data('https://books.toscrape.com/')
print('Time taken (normal function):', time.time() - start_time)

# Get the time it took for a memoized function (LRU cache)
start_time = time.time()
get_html_data_lru('https://books.toscrape.com/')
print('Time taken (memoized function with LRU cache):', time.time()
```

Source

System Design Examples



Source

Proxy

…Don't you think **you meet** this concept **everyday**?

Your Journey

01

Introduction
&
Strategy pattern

02

Observer
&
Decorator

03

Creational
Patterns

04

Command
&
Adapter

05

Facade
&
Template Method

06

Iterator
&
Composite

07

State
&
Proxy

08

...

Your Journey

09

...

...

10

11

Surprise!

What I've implemented

01

Strategy pattern ✅

Observer ❌
&
Decorator ✅

02

03

Creational
Patterns ✅

Command ❌
&
Adapter ❓❌

04

Facade ❓ ✅
&
Template Method ❓✅

05

06

Iterator ❌✅❓
&
Composite ❌

07

State ❌
&
Proxy ❌

08

...

Advanced patterns

Irina's personal check boxes:

- Chain of responsibility ✅
- Flightweight ❌
- Bridge ❌
- Visitor ❌
- Mediator ❌
- Memento ❌
- Interpreter pattern ❓ ✅

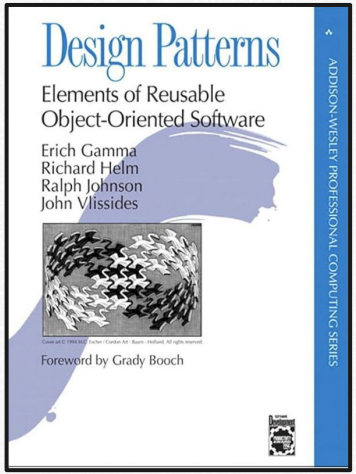
Resources

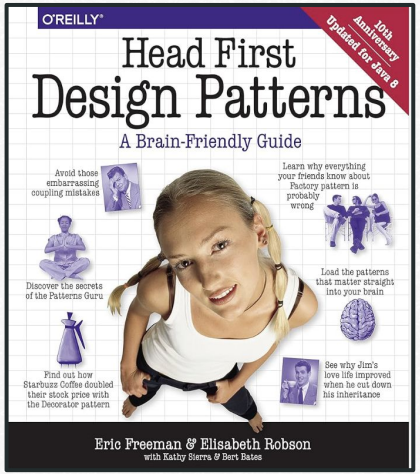
Books:

- Classic GOF Pattern
- Racoon Book
- Head First Design Patterns

Examples:

- Presentation (in Slack)





Thank you!